

ASP.NET Core in Action

Chapter 10: Service configuration with dependency injection

19 February 2024

Q1. Which one is responsible for creating instances of services that are passed as dependency injection?

- A. View Engine
- B. IoC container
- C. Model binder
- D. Controller

Answer: B

[Page 298]

Also, DI container also known as IoC container]

Q2. Which method of the IServiceCollection is used to register a dependency service to the container?

- A. Register*
- B. Add*
- C. Create*
- D. Use*

Answer: B

[Page 306]

Q3. What information do you need to provide to register a DI service to the container?

- A. Service type
- B. Implementation type
- C. Lifetime
- D. Constructor type

Answer: A, B, C

[Page 306]

Q4. Which one is not a valid lifetime of DI service?

- A. transient
- B. singleton
- C. singlecall
- D. scoped

Answer: C

[Page 306]

transient, singleton, or scoped]

Q5. Which of the following are valid method call to register a DI service to the container?

- A. AddScoped<TService>
- B. AddScoped<TService, TImplementation>
- C. AddScoped<TService> where type-constraints
- D. AddScoped<TService, TImplementation> where type-constraints

Answer: A, B

[Page 306]

Q6. You have a service like below

```
public class MailServerSettings
{
    public MailServerSettings(string host, int port){ Host=host; Port=port;}
```

```

        public string Host { get; }
        public int Port { get; }
    }

```

Which one of the following codes correctly register the service to the DI container?

- A. `services.AddSingleton<MailServerSettings>()`
- B. `services.AddSingleton<MailServerSettings, MailServerSettings>()`
- C. `services.AddSingleton(new MailServerSettings`
`{`
 `Host= "...", Port= 25`
`})`
- D. `services.AddSingleton(new MailServerSettings())`

Answer: C

[Page 308

- A. Class has no default constructor
- B. No interface of the class
- C. Ok
- D. Class has no default constructor]

Q7. You have a service like below

```

public class MailServerSettings
{
    public MailServerSettings(string host, int port){ Host=host; Port=port;}
    public string Host { get; }
    public int Port { get; }
}

```

Which one of the following codes correctly register the service to the DI container?

- A. `services.AddScoped<MailServerSettings>()`
- B. `services.AddScoped<MailServerSettings, MailServerSettings>()`
- C. `services.AddScoped(new MailServerSettings`
`{`
 `Host= "...", Port= 25`
`})`
- D. `services.AddScoped(provider => new MailServerSettings`
`{`
 `Host= "...", Port= 25`
`});`

Answer: D

[Page 309

Scoped here]

Q8. You have the following generic class

```

public interface IRepository<T> {}
public class DbRepository<T> : IRepository<T>{}

```

Which one of the following codes correctly register the service to the DI container?

- A. `services.AddScoped<DbRepository<T>>>()`
- B. `services.AddScoped<IRepository<T>, DbRepository<T>>>()`
- C. `services.AddScoped<IRepository<>, DbRepository<>>>()`
- D. `services.AddScoped<typeof(IRepository<>), typeof(DbRepository<>>>())`

Answer: D

[Page 341,342]

Q9. Besides the constructor of the controller, at what other locations can you use dependency injection?

- A. Action methods
- B. Page handler methods
- C. View templates
- D. Property setters in controller class

Answer: A, B, C

[Page 315]

Q10. Which attribute should you use to inject a dependency to an action method?

- A. [Inject]
- B. [FromServices]
- C. [FromBody]
- D. [Bind]

Answer: B

[Page 317]

Q11. Which directive should you use to inject a dependency to a view template?

- A. @imoort
- B. @using
- C. @inject
- D. @page

Answer: C

[Page 318]

Q12. What is the lifetime of a service?

- A. how long an instance of a service should be in activated before deactivating it
- B. how long an instance of a service should live in a container before it creates a new instance
- C. how long an instance of a service should the container keep it in a pool
- D. dictates how many instances of the service the container should create

Answer: B

[Page 318]

Q13. What does the transient lifetime of a service mean?

- A. Every time a service is requested, a new instance is created
- B. Within a scope, all requests for a service will give you the same object
- C. All requests for a service get the same instance of the service, no matter which scope
- D. None of the above

Answer: A

[Page 318]

Q14. What does the Scoped lifetime of a service mean?

- A. Every time a service is requested, a new instance is created
- B. Within a scope, all requests for a service will give you the same object
- C. All requests for a service get the same instance of the service, no matter which scope
- D. None of the above

Answer: B

[Page 318]

Q15. What does the Singleton lifetime of a service mean?

- A. Every time a service is requested, a new instance is created
- B. Within a scope, all requests for a service will give you the same object
- C. All requests for a service get the same instance of the service, no matter which scope
- D. None of the above

Answer: C

[Page 318]